

LAYORA

Autonomous Student Productivity Suite

Web Technology Mini Project Report

Full-Stack Next.js & Firebase Application

DEVELOPMENT STACK TELEMETRY

Frontend Frame: React 19, Next.js 15, Tailwind v4
Data Storage: Firebase Firestore & Zustand local-persist
Third-Party: Clerk Identity, Google APIs, Groq AI Engine
Local Verification: Node.js v22.11.0, Python 3.14.2
Remote Deployment Host: Vercel Cloud Serverless

1. Project Basics

- **Name:** LAYORA
- **Tagline:** Autonomous Student Productivity Suite
- **Problem Statement:** Traditional student schedules are rigid, hard to balance, lead to cognitive burnout, and fail to adapt to individual credits, priorities, sleep cycles, and extracurricular activities. Students lack an integrated dashboard that combines study materials, tasks with active timers, and an AI mentor to plan their schedules and offer personalized academic guidance.
- **Main Objective / Purpose:** To provide an autonomous, full-stack student assistant that optimizes study cycles, generates dynamic schedules based on subjects and courses, provides global stopwatch timers to track study sessions, connects with Google Calendar/Drive, and integrates a chatbot for proactive academic mentorship.

2. Technologies Used

- **Frontend Core:** HTML5 (React JSX), TypeScript, Next.js 15 (App Router, React 19)
- **Styling & Animation:** Tailwind CSS v4, Custom CSS Glassmorphism effects, Framer Motion
- **State Management:** Zustand State Library (with localStorage persistence middleware)
- **Backend API Core:** Next.js serverless API routes
- **Database & Files:** Firebase Firestore & Cloud Storage (fallback to local Zustand persistent storage)
- **Authentication:** Clerk Auth Portal (Secure User Identity with OAuth/SSO)
- **AI Inference:** Groq Llama-3.1-8b-instant endpoint for Chatbot & Schedule compiler
- **Third-Party Sync:** Google Calendar API (timetable sync), Google Drive API (notes uploads)
- **Tools & Servers:** VS Code (IDE), Git (Version Control), Vercel (Hosting Server)

3. Hardware & Software Requirements

Hardware Requirements (Client & Developer)

- **Client CPU:** Dual-Core 1.6 GHz or faster
- **Client Memory:** 4 GB RAM minimum
- **Client Disk Space:** 500 MB minimum (for browser cache & state)
- **Developer CPU:** Quad-Core 2.4 GHz or higher (Intel Core i5 / Ryzen 5 equivalent)
- **Developer Memory:** 8 GB RAM minimum (16 GB recommended)
- **Developer Storage:** 10 GB SSD space (for node_modules and builds)

Software Requirements

- **Supported OS:** Windows 10/11, macOS Catalina or higher, Linux Ubuntu/Debian
- **Web Browsers:** Google Chrome, Mozilla Firefox, Safari, Microsoft Edge (ES6 compatible)
- **Node.js Environment:** v18.0.0 or higher (Current local environment verified on v22.11.0)
- **NPM Package Manager:** v9.0.0 or higher (Current local environment verified on v10.9.0)

4. Features / Modules

- **Secure Gateway (Auth):** User login, signup, and session controls handled securely via Clerk integration.
- **Onboarding Steps:** A 7-step wizard capturing student sleeping hours, academic subjects, difficulty rankings, syllabus PDFs, activities, website goals, and active courses.
- **Dashboard Home:** Highlights metrics like streak count, study hours, active daily schedule timeline, upcoming tomorrow agenda, course tracks, and website launchers.
- **Proactive AI Engine:** Runs background analysis on subjects/deadlines to recommend the "Next Best Task", priority subjects, and gives mentor tips.
- **Weekly Planner Grid:** A clean interactive timetable. Renders scheduled slots (lectures, breaks, studies, activities) and exports to Google Calendar.
- **Stopwatch Session Timer:** A global task timer running in the top bar to track real study minutes and update task statistics upon completion.
- **Resource Vault:** Sorts uploaded academic notes and uploads them directly to the student's personal Google Drive.
- **Settings Console:** Accents manager, time formatting toggles, custom AI keys settings, and database reset switches.

5. Design Details

User Flow Detail

1. Landing: User attempts to access Layora. If not logged in, the Clerk Auth module intercepts the request and forces login.
2. Registration: User registers/signs in via OAuth (Google) or email/password. Clerk sets the local session cookie.
3. Onboarding: First-time users navigate through the 7-step wizard. Inputs are stored in the local Zustand store state.
4. Scheduler Compilation: On saving the onboarding setup, the system generates a custom week schedule. If Groq API is enabled, a server route generates the schedule via LLM; otherwise, the local solver resolves overlaps and outputs the schedule.
5. Workspace: The user interacts with the Dashboard: completes tasks, records study blocks with the stopwatch, uploads documents, view statistics charts, and asks questions in the AI Chatbot.

Database Sync Flow

To ensure offline compatibility and state durability, Layora uses a "Zustand Local-First + Firebase Sync" architecture:

- **Offline Write:** Zustand captures all state updates (tasks toggles, settings) and immediately saves them to browser LocalStorage.
- **Firebase Sync Listener:** If Firebase configuration is active, the SyncProvider component listens to Firestore path `user_states/{uid}`. Every Zustand store change is debounced by 200ms and written to Firestore. If the document is modified on another device, Firestore pushes updates back, merging lists safely.

6. Code Snippets

Snippet: Overlap Resolution Algorithm

```
// Snippet 1: Overlap Resolution Algorithm in src/lib/aiService.ts
export function resolveScheduleOverlaps(schedule: TimetableBlock[]): TimetableBlock[] {
  const timeToMin = (t: string) => {
    const [h, m] = t.split(':').map(Number);
    return h * 60 + m;
  };
  const minToTime = (m: number) => {
    const h = Math.floor(m / 60) % 24;
    const min = Math.floor(m % 60);
    return `${h.toString().padStart(2, '0')}:${min.toString().padStart(2, '0')}`;
  };
  const getBlockPriority = (block: TimetableBlock, index: number) => {
    if (block.type === 'class') return 100000000 + index;
    if (block.id.startsWith('custom-') || block.id.startsWith('ai-')) return 1000000 + index;
    return index;
  };
  const adjustedSchedule: TimetableBlock[] = [];
  for (let day = 0; day <= 6; day++) {
    const dayBlocks = schedule.filter(b => b.day === day);
    const sorted = dayBlocks.map((b, i) => ({ block: b, p: getBlockPriority(b, i) })).sort((a, b) => b.p - a.p);
    const occupied: { start: number; end: number }[] = [];
    sorted.forEach(({ block }) => {
      const segments = adjustBlockAgainstOccupied(block, occupied);
      segments.forEach((seg) => {
        adjustedSchedule.push(seg);
        occupied.push({ start: timeToMin(seg.start), end: timeToMin(seg.end) });
      });
    });
  }
  return adjustedSchedule.sort((a, b) => a.day !== b.day ? a.day - b.day : a.start.localeCompare(b.start));
}
```

Snippet: Proactive AI Recommender Route

```
// Snippet 2: Proactive AI API Endpoint in src/app/api/ai/proactive/route.ts
export async function POST(req: NextRequest) {
  try {
    const { currentTasks, currentSubjects, currentRoutine } = await req.json();
    const systemPrompt = `Analyze workload. Return JSON matching:
    {
      "nextBestTask": "Title of the task they should start right now",
      "urgentSubject": "Subject needing attention",
      "recommendedDuration": 45,
      "workloadWarning": "Warning sentence",
      "mentorAdvice": "Coaching note"
    }`;
    const groqRes = await fetch('https://api.groq.com/openai/v1/chat/completions', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        Authorization: `Bearer ${process.env.GROQ_API_KEY}`,
      },
      body: JSON.stringify({
        model: 'llama-3.1-8b-instant',
        response_format: { type: 'json_object' },
        messages: [{ role: 'system', content: systemPrompt }, { role: 'user', content: 'Generate recommendations.' }],
      })),
  } catch (error) {
    console.error('Error in proactive AI route:', error);
  }
}
```

```
    if (groqRes.ok) {  
      const data = await groqRes.json();  
      return NextResponse.json(JSON.parse(data.choices[0].message.content));  
    }  
    return NextResponse.json({ nextBestTask: "Review pending tasks", urgentSubject: "Dashboard", recommendedDuration:  
60 });  
  } catch (error: any) {  
    return NextResponse.json({ error: error.message }, { status: 500 });  
  }  
}
```

Snippet: Firebase Real-Time State Syncer

```
// Snippet 3: SyncProvider Subscriptions in src/components/SyncProvider.tsx
const unsubscribe = useStore.subscribe((state) => {
  if (!isHydrated.current) return;
  const { user, subjects, resources, activities, tasks, timetable } = state;
  const stateToSave = { user, subjects, resources, activities, tasks, timetable };
  const serialized = JSON.stringify(stateToSave);
  if (serialized === lastSavedSerializedRef.current) return;
  pendingStateRef.current = state;
  if (syncTimeoutRef.current) clearTimeout(syncTimeoutRef.current);
  const performSync = async () => {
    if (suppressSync) return;
    try {
      inFlightWrites.current++;
      const docRef = doc(db, 'user_states', user.id);
      await setDoc(docRef, { state: sanitizeStateForFirestore(stateToSave), updated_at: new Date().toISOString() });
      lastSavedSerializedRef.current = serialized;
      ignoreSnapshotUntilRef.current = Date.now() + 3000;
    } catch (err: any) {
      console.error('Failed to sync:', err);
    } finally {
      inFlightWrites.current--;
    }
  };
  const isImmediate = JSON.stringify(subjects) !== JSON.stringify(lastSavedSubjectsRef.current);
  if (isImmediate) performSync();
  else syncTimeoutRef.current = setTimeout(performSync, 200);
});
```

7. Testing

Manual and logic unit testing was carried out for the main modules:

ID	Module	Input	Expected Output	Status
TC-01	Authentication Gateway	Anonymous user loads /dashboard	Redirects to /login, renders Clerk gateway portal	PASS
TC-02	Onboarding Wizard	Enters subjects + routine sleep-times	Zustand updates, calls generateLocalWeeklySchedule	PASS
TC-03	Planner Overlaps	Add manual block during lecture hours	Lecture block overrides, study block scheduled later	PASS
TC-04	Stopwatch Study Timer	Click "Start Session" on math class card	Global nav stopwatch ticks. Completing adds actualMinutes	PASS
TC-05	Cloud Sync Service	Subject deleted offline, reconn internet	Firebase documents sync, deleted subjects removed on server	PASS

8. Screen Layouts & Wireframes

As Layora features a high-fidelity dark glassmorphic design system, here is how the core screens render:

- **Sign In screen (/login):** A cosmic-black backdrop (#070709) with ambient cyan and purple neon glow orbs. The central interactive login panel contains secure email/password fields and quick Google OAuth actions.
- **Onboarding Portal (/onboarding):** Renders a smooth sliding panel layout with step numbers and progress trackers at the top. Provides styled selectors, timers, and list containers.
- **Core Dashboard (/dashboard):** Organizes study stats (total hours, streak fire, pending logs, task ratios) at the top. Displays daily timetable timeline feeds with quick "Start Session" triggers on study blocks.
- **AI chatbot & settings (/dashboard/settings):** Divided panel displaying accent switches (purple, blue, pink, emerald), time configuration options, and custom API key binds to connect live models.

9. Conclusion

What Layora Achieves

Layora successfully integrates multiple student academic management workflows into a single interface. By combining a scheduling solver with stopwatch trackers and AI guidance, it converts passive schedule tracking into active study progress.

Who Benefits from It

Students looking to maintain study streaks, build continuous algorithmic practice habits (LeetCode daily integrations), organize reference documents, and sync academic schedules with standard utilities.

Future Scope

- **Social study lobbies:** Allowing students to share timetable templates and host shared focus periods.
- **Interactive Browser Ext:** A companion extension that blocks access to non-essential sites during active study blocks.
- **Natural voice commands:** Leveraging OpenAI whisper APIs for completely voice-controlled navigation.